

fn make_int_to_bigint_threshold

Michael Shoemate (@Shoeboxam)

This proof resides in “**contrib**” because it has not completed the vetting process.

Proves soundness of the implementation of `make_int_to_bigint_threshold` in `mod.rs` at commit `f5bb719` (outdated¹).

1 Hoare Triple

Precondition

Compiler-verified

- Generic TK implements trait `Hashable`
- Generic TV implements trait `Integer` and `SaturatingCast<IBig>`
- Const-generic P is of type `usize`
- Generic QI implements trait `Number`

Caller-verified

None

Pseudocode

```
1 def make_int_to_bigint_threshold(  
2     input_space: tuple[  
3         MapDomain[AtomDomain[TK], AtomDomain[TV]],  
4         LOPInfDistance[P, AbsoluteDistance[QI]],  
5     ],  
6 ) -> Transformation[  
7     MapDomain[AtomDomain[TK], AtomDomain[TV]],  
8     MapDomain[AtomDomain[TK], AtomDomain[IBig]],  
9     LOPInfDistance[P, AbsoluteDistance[QI]],  
10    LOPInfDistance[P, AbsoluteDistance[RBig]],  
11 ]:  
12     input_domain, input_metric = input_space  
13  
14     def stability_map(d_in):  
15         l0, lp, li = d_in  
16         lp = UBig.try_from(lp)  
17         li = UBig.try_from(li)  
18         return l0, lp, li  
19  
20     return Transformation.new(  
21         input_domain, input_metric, stability_map
```

¹See new changes with `git diff f5bb719..e01f737a rust/src/measurements/noise_threshold/nature/integer/mod.rs`

```

21     input_domain,
22     MapDomain( #
23         key_domain=input_domain.key_domain,
24         value_domain=AtomDomain.default(IBig),
25     ),
26     Function.new(
27         lambda x: {k: IBig.from_(v) for k, v in x.items()}
28     ), #
29     input_metric,
30     LOPI.default(),
31     StabilityMap.new_fallible(stability_map),
32 )

```

Postcondition

Theorem 1.1. For every setting of the input parameters (`input_space`, `TK`, `TV`, `P`, `QI`) to `make_int_to_bigint_threshold` such that the given preconditions hold, `make_int_to_bigint_threshold` raises an error (at compile time or run time) or returns a valid transformation. A valid transformation has the following properties:

1. (Data-independent runtime errors). For every pair of members x and x' in `input_domain`, `invoke(x)` and `invoke(x')` either both return the same error or neither return an error.
2. (Appropriate output domain). For every member x in `input_domain`, `function(x)` is in `output_domain` or raises a data-independent runtime error.
3. (Stability guarantee). For every pair of members x and x' in `input_domain` and for every pair (d_in, d_out) , where d_in has the associated type for `input_metric` and d_out has the associated type for `output_metric`, if x, x' are d_in -close under `input_metric`, `stability_map(d_in)` does not raise an error, and `stability_map(d_in) = d_out`, then `function(x), function(x')` are d_out -close under `output_metric`.

Proof. By the definition of the function on line 28, and since `IBig.from` is infallible, the function is infallible, meaning that the function cannot raise data-dependent errors. The function also always returns a hashmap whose keys are un-changed, and values are `IBigs`, meaning the output of the function is always a member of the output domain, as defined on line 22. Finally, the function is 1-stable, because the real values of the numbers remain un-changed, meaning the distance between adjacent inputs always remains the same, satisfying the stability property. $\square$